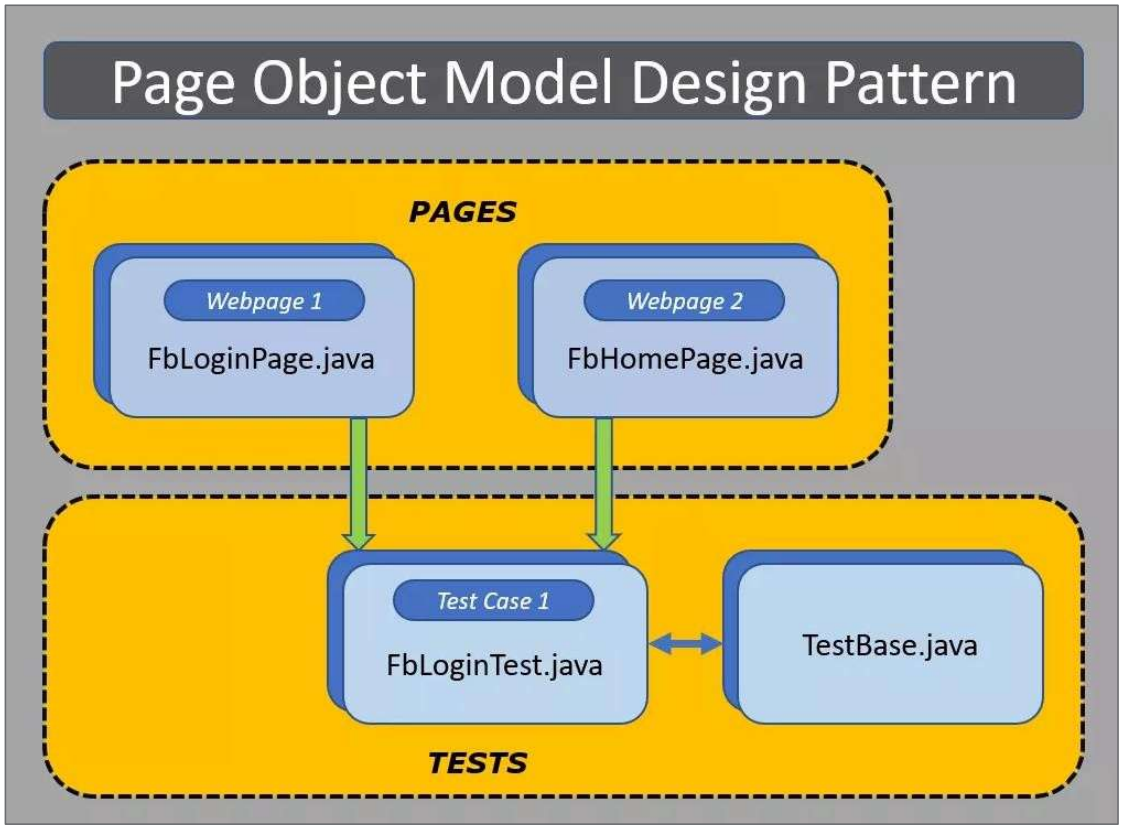
Test Automation Frameworks

Module 1
Page Object Model Pattern

Page Object Model Design Pattern

What are Page Object Model?



Page Object Model Design Pattern –Contd..

Why Page Object Model?

- Duplication of Code
- Less Time
- Code Maintenance

Page Object Model Design Pattern –Contd..

Complete Example for Page Object Model:

TestCase: Go to Google site

Step1: Go to Google site

Step 2: In the Home Page Check the title of the page

Step 3: Type the text “Hello” in search box

Step 4: Click on the Search Button to Search in google

Here we are dealing with one page – **Google Home page**

Accordingly , we create one POM Class

Page Object Model Design Pattern –Contd..

Googlehomepage.java

```
import org.openqa.selenium.By;

import org.openqa.selenium.WebDriver;

public class Googlehomepage
 {
    WebDriver driver;
    By searchbox = By.name("q");
    By searchbtn= By.name("btnK");
    By titleText= By.tagName("title");
    public Googlehomepage(WebDriver driver)
        {
        this.driver = driver;
        }
    //Set  nam in textbox
    public void setContent(String Name)
        {
        driver.findElement(searchbox).sendKeys(Name);
        }
    //Click on Search Button
    public void clickLogin()
        {
            driver.findElement(searchbtn).click();
        }
    //Get the title of Google Page
    public String getTitle()
        {
     return   driver.findElement(titleText).getText();
        }
}
```

Page Object Model Design Pattern –Contd..

Testgooglehomepage.java (TestCase)

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.chromeDriver;
import org.testng.annotations.Test;

import pom.Googlehomepage;

public class Testgooglehomepage
{
@Test
public void validatePage()throws InterruptedException
{
//Initialize ChromeDriver
System.setProperty("webdriver.chrome.driver","Browserdriver folder path");
WebDriver driver=new ChromeDriver();
driver.get("https://www.google.co.in");

driver.manage().window().maximize();
Thread.sleep(5000);
Googlehomepage gpage=new Googlehomepage(driver);
gpage.getTitle();
gpage.setContent("Hello");
gpage.clickLogin();
driver.close();
}
```

Page Object Model Design Pattern –Contd..

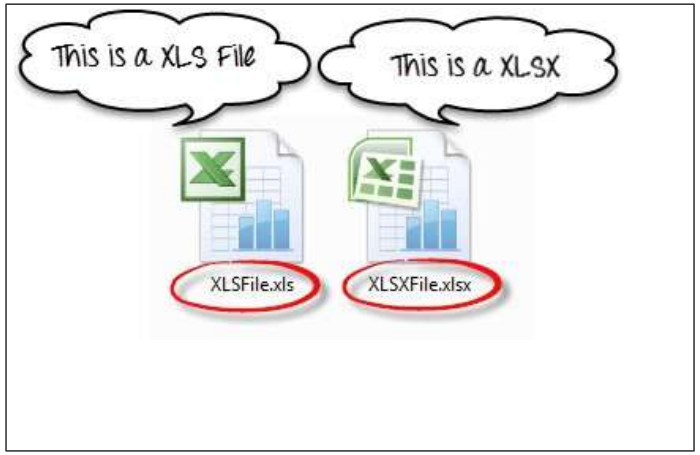
Advantages of Page Object Model Design Pattern:

- This will keep the code clean and easy to understand and maintain.
- The Page Object approach makes automation framework in a testing programmer friendly, more durable and comprehensive.
- The purposes with different frameworks like you will be able to integrate this repository with other tools like **JUnit/NUnit/PhpUnit** as well as with **TestNG/Cucumber**/etc.

Module 2
Data Driven Test Framework

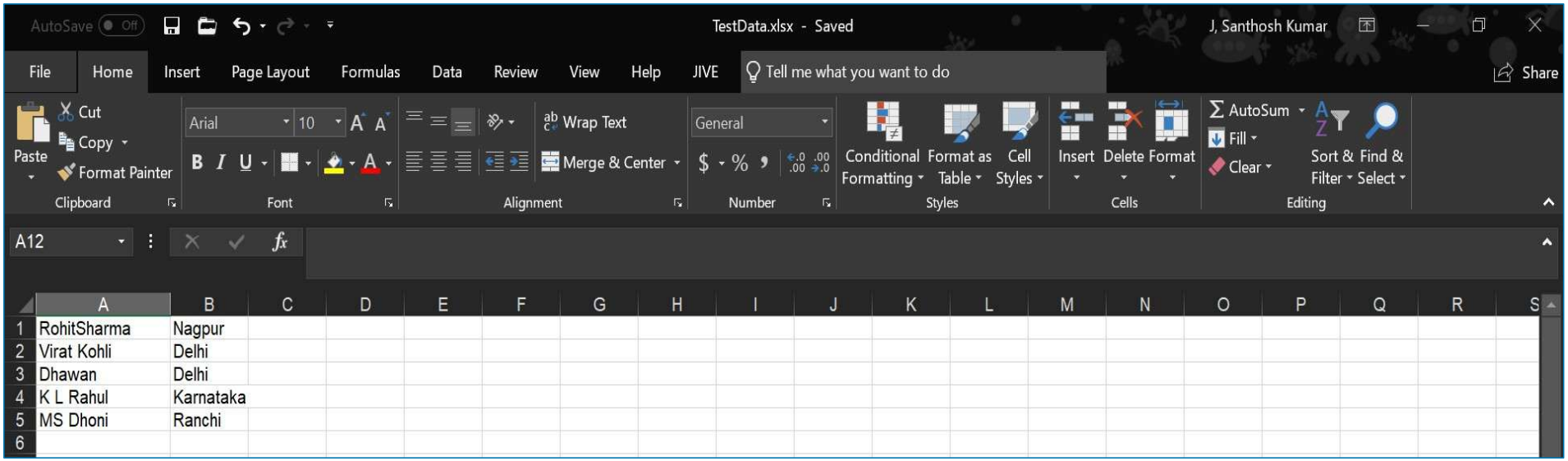
Data Driven Test Framework

- **In** data driven framework all of our test data is generated from some external files like Excel, CSV, XML or some database table.
- To read or write an Excel, Apache provides a very famous library POI. This library is capable enough to read and write both **XLS** and **XLSX** file format of Excel.
- To read **XLS** files, an **HSSF** implementation is provided by POI library.
- To read **XLSX, XSSF** implementation of **POI library**



Data Driven Test Framework- Contd..

Create New Excel File as “TestData.xlsx”



After Creating the Excel File we can perform two Operations.

1. Reading the data from the excel
2. Writing the data into excel

Data Driven Test Framework- Contd..

Code for Reading the Data From Excel

```
package excelExportAndFileIO;
import java.io.File;
import java.io.FileInputStream;
import java.io.IOException;
import org.apache.poi.hssf.usermodel.HSSFWorkbook;
import org.apache.poi.ss.usermodel.Row;
import org.apache.poi.ss.usermodel.Sheet;
import org.apache.poi.ss.usermodel.Workbook;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;

public class ReadExcelFile {

    public void readExcel(String fileName,String sheetName) throws IOException{
    //Create an object of File class to open xlsx file
    File file = new File("File path to read the file");
    //Create an object of FileInputStream class to read excel file
    FileInputStream inputStream = new FileInputStream(file);
    Workbook workbook = null;
    //Find the file extension by splitting file name in substring  and getting only extension name
    String fileExtensionName = fileName.substring(fileName.indexOf("."));
    //Check condition if the file is xlsx file
    if(fileExtensionName.equals(".xlsx")){
    //If it is xlsx file then create object of XSSFWorkbook class
    workbook = new XSSFWorkbook(inputStream);
    }
    //Check condition if the file is xls file
    else if(fileExtensionName.equals(".xls")){
        //If it is xls file then create object of HSSFWorkbook class
        workbook = new HSSFWorkbook(inputStream);
    }
    //Read sheet inside the workbook by its name
    Sheet sheet = workbook.getSheet(sheetName);
    //Find number of rows in excel file
    int rowCount = sheet.getLastRowNum()-sheet.getFirstRowNum();
    //Create a loop over all the rows of excel file to read it
    for (int i = 0; i < rowCount+1; i++) {
        Row row = sheet.getRow(i);
        //Create a loop to print cell values in a row
        for (int j = 0; j < row.getLastCellNum(); j++) {
            //Print Excel data in console
            System.out.print(row.getCell(j).getStringCellValue()+"|| ");
        }
        System.out.println();
    }
    }

    //Main function is calling readExcel function to read data from excel file
    public static void main(String...strings) throws IOException{
    //Create an object of ReadExcelFile class
    ReadExcelFile objExcelFile = new ReadExcelFile();
    //Call read file method of the class to read data
    objExcelFile.readExcel("TestData.xlsx","Data");
    }
}
```

Data Driven Test Framework- Contd..

```
package excelExportAndFileIO;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import org.apache.poi.hssf.usermodel.HSSFWorkbook;
import org.apache.poi.ss.usermodel.Cell;
import org.apache.poi.ss.usermodel.Row;
import org.apache.poi.ss.usermodel.Sheet;
import org.apache.poi.ss.usermodel.Workbook;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;

public class WriteExcelFile {
    public void writeExcel(String fileName,String sheetName,String[] dataToWrite) throws IOException{
        //Create an object of File class to open xlsx file
        File file =    new File("FilePath to Open the Excel File");
        //Create an object of FileInputStream class to read excel file
        FileInputStream inputStream = new FileInputStream(file);
        Workbook workbook = null;
        //Find the file extension by splitting  file name in substring and getting only extension name
        String fileExtensionName = fileName.substring(fileName.indexOf("."));
        //Check condition if the file is xlsx file
        if(fileExtensionName.equals(".xlsx")){
        //If it is xlsx file then create object of XSSFWorkbook class
        workbook = new XSSFWorkbook(inputStream);
        }
        //Check condition if the file is xls file
        else if(fileExtensionName.equals(".xls")){
            //If it is xls file then create object of XSSFWorkbook class
            workbook = new HSSFWorkbook(inputStream);
        }
    //Read excel sheet by sheet name
    Sheet sheet = guru99Workbook.getSheet(sheetName);
    //Get the current count of rows in excel file
    int rowCount = sheet.getLastRowNum()-sheet.getFirstRowNum();
```

```
    //Get the first row from the sheet
    Row row = sheet.getRow(0);
    //Create a new row and append it at last of sheet
    Row newRow = sheet.createRow(rowCount+1);
    //Create a loop over the cell of newly created Row
    for(int j = 0; j < row.getLastCellNum(); j++){
        //Fill data in row
        Cell cell = newRow.createCell(j);
        cell.setCellValue(dataToWrite[j]);
    }
    //Close input stream
    inputStream.close();
    //Create an object of FileOutputStream class to create write data in excel file
    FileOutputStream outputStream = new FileOutputStream(file);
    //write data in the excel file
    workbook.write(outputStream);
    //close output stream
    outputStream.close();
    }
    public static void main(String...strings) throws IOException{
        //Create an array with the data in the same order in which you expect to be filled in excel file
        String[] valueToWrite = {"Bumrah","Gujarat"};
        //Create an object of current class
        WriteExcelFile objExcelFile = new WriteExcelFile();
        //Write the file using file name, sheet name and the data to be filled
        objExcelFile.writeExcel(System.getProperty("TestData.xlsx","Data",valueToWrite);
    }
}
```

Module 3
TestNG Framework

TestNG Framework

What is TestNG?

- TestNG is an automation testing framework in which NG stands for "Next Generation". TestNG is inspired from JUnit which uses the annotations (@).
- Using TestNG you can generate a proper report, and you can easily come to know how many test cases are passed, failed and skipped and you can execute failed test case separately.

Why Use TestNG with Selenium?

- Default Selenium tests do not generate a proper format for the test results. Using TestNG we can generate test results.

Annotations in TestNG:

@Test - The annotated method is a part of a test case

@BeforeTest – The annotated method will be run before any test method belonging to the classes inside the tag is run.

@AfterTest – The annotated method will be run after all the test methods belonging to the classes inside the tag have run.

@BeforeMethod - The annotated method will be run before each test method.

@AfterMethod - The annotated method will be run after each test method.

@BeforeClass – The annotated method will be run before the first test method in the current class is invoked.

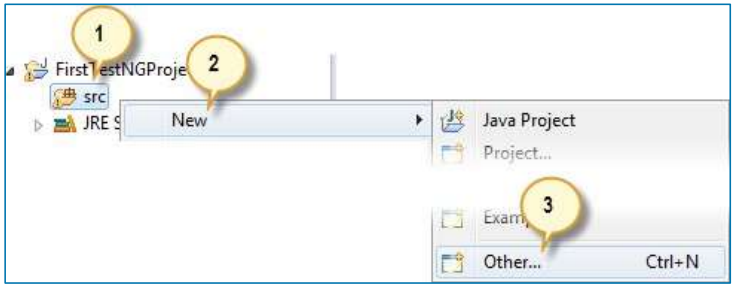
@AfterClass – The annotated method will be run after all the test methods in the current class have been run.

TestNG Framework-Contd..

Creating a New TestNG Test File

Step 1: Create the new project name as “FirstTestNGProject”

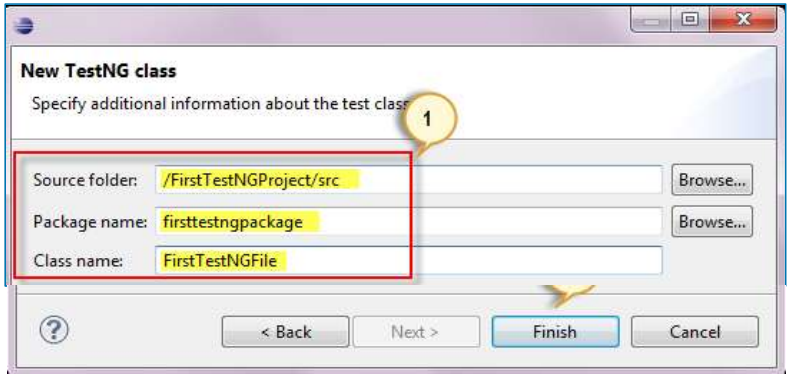
Step2: Right-click on the "src" package folder then choose New > Other…



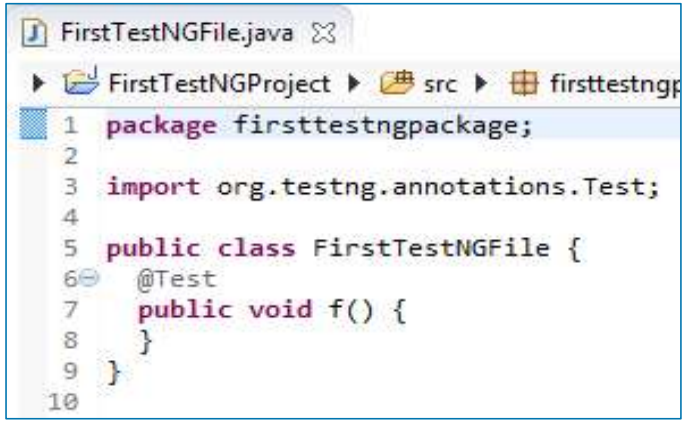
Step 3: Click on the TestNG folder and select the "TestNG class" option. Click Next.

TestNG Framework-Contd..

Step 4: Type the values indicated below on the appropriate input boxes and click Finish. Notice that we have named our Java file as "**FirstTestNGFile**".



Step 5: Eclipse should automatically create the template for our TestNG file shown below.



TestNG Framework-Contd..

First Test Case Program:

- We used the @Test annotation. **@Test is used to tell that the method under it is a test case**. In this case, we have set the verifyHomepageTitle() method to be our test case, so we placed an '@Test' annotation above it.

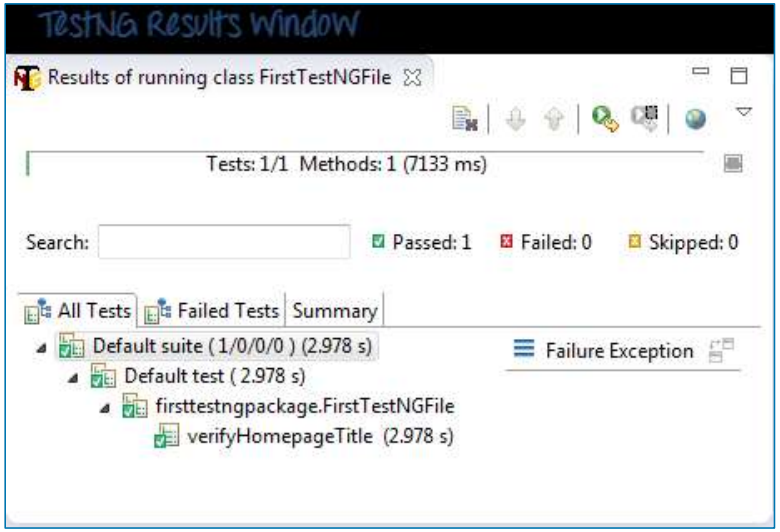
```
package firsttestng;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.Test;

public class FirstTestNGFile {
  @Test
  public void verifyHomepageTitle()
  {
      System.setProperty("webdriver.chrome.driver","F:\\Selenium Support\\chromedriver.exe");

        WebDriver driver=new ChromeDriver();
        driver.get("https://google.com");
        driver.close();
  }
}
```

TestNG Framework-Contd..

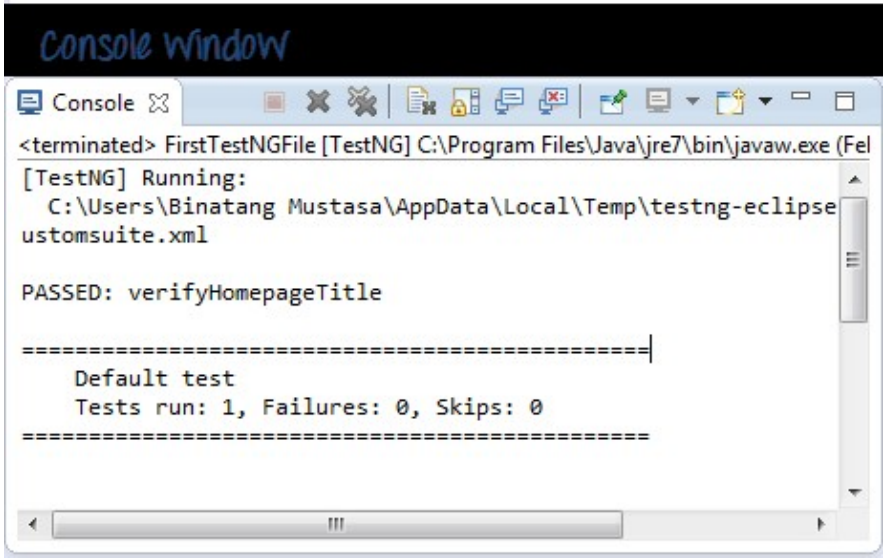
Running the Test

To run the test, simply run the file in Eclipse as you normally do. Eclipse will provide two outputs – one in the Console window and the other on the TestNG Results window.

Test Results Window



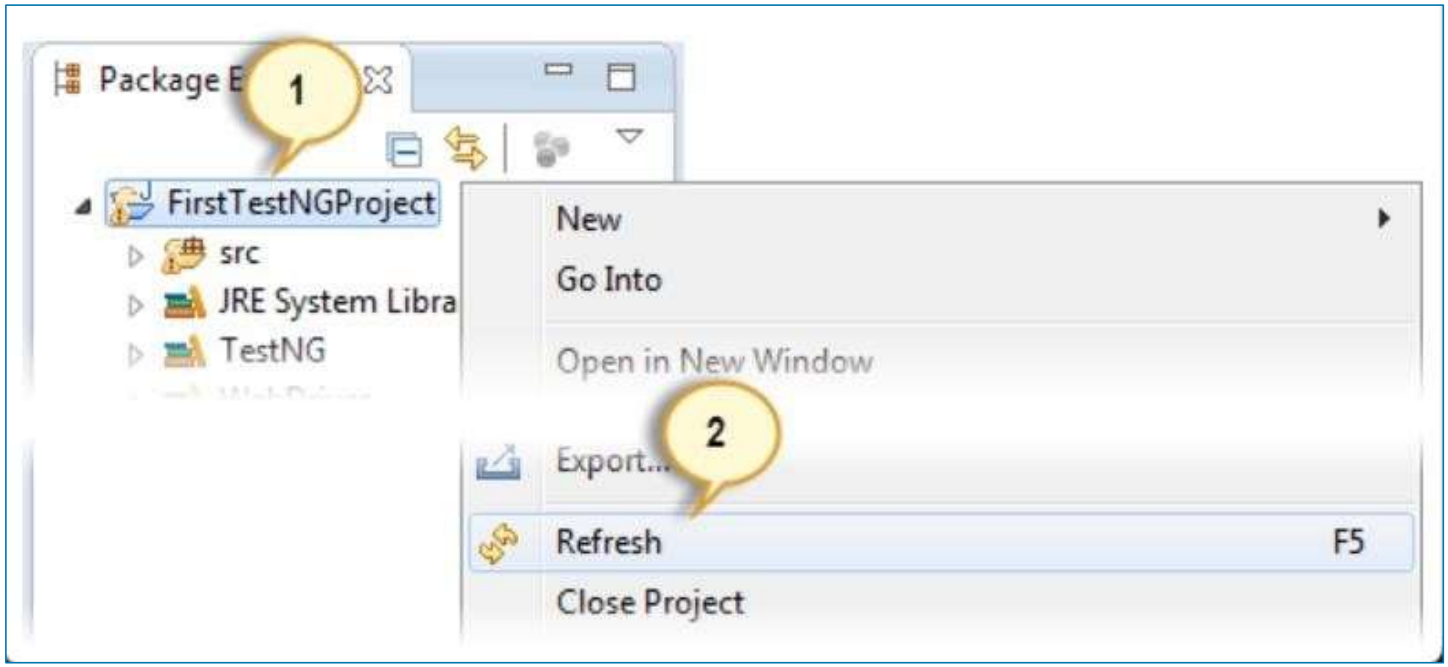
Console Window



TestNG Framework-Contd..

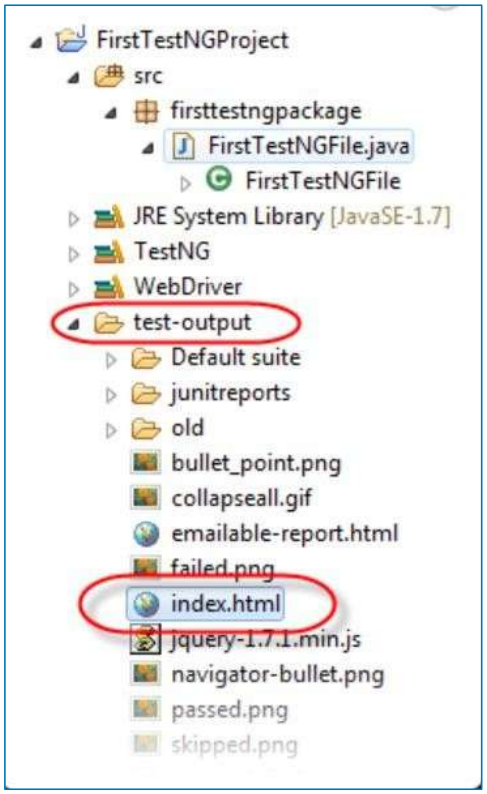
Generating HTML Reports

Step 1: After running our FirstTestNGFile that we created in the previous section, right-click the project name (FirstTestNGProject) in the Project Explorer window then click on the "Refresh" option.



TestNG Framework-Contd..

Step 2: Notice that a "test-output" folder was created. Expand it and look for an index.html file. This HTML file is a report of the results of the most recent test run.



TestNG Framework-Contd..

Step 3: Double-click on that index.html file to open it within Eclipse's built-in web browser. You can refresh this page any time after you rerun your test by simply pressing F5 just like in ordinary web browsers.